LEAD AI ENGINEER

BACKEND DEVELOPMENT & APIs

10 exact questions. Practical answers.

Interview perspectives, examples and memory tips.

01 Diagnose the production symptom

02 Isolate the bottleneck with evidence

03 Explain the design trade-offs

04 Validate the fix with measurable outcomes

Save this carousel for your Lead AI Engineer interview.

Q1. Your FastAPI application works locally but becomes slow when 500 concurrent users access it. How would you diagnose and resolve the bottleneck?

HOW I WOULD ANSWER

- Confirm the symptom: concurrency is not requests/sec; reproduce realistic traffic with k6 or Locust and capture p50/p95, throughput and error rates.
- Break latency into request queue, FastAPI handler, database, external LLM/API, and serialization using distributed traces.
- Inspect CPU, event-loop blocking, worker count, connection pools, memory and Azure OpenAI throttling; optimize the measured bottleneck.
- Load-test after changes; tune concurrency, apply backpressure and scale AKS only when utilization or queue depth warrants it.

PRACTICAL EXAMPLE

Example: k6 load test + OpenTelemetry traces; inspect HPA and database pool wait time.

INTERVIEW PERSPECTIVE

State the SLO and workload assumptions first. Explain why scaling pods cannot fix a shared provider quota.

QUICK MEMORY TIP

MEASURE > LOCATE > FIX > RETEST

AVOID

Adding workers or pods before locating the bottleneck.

Q2. An API endpoint takes 12 seconds to respond, but the database query takes only 100 ms. How would you isolate the latency?

HOW I WOULD ANSWER

- Add a correlation/request ID and trace every span: middleware, authentication, dependency injection, retrieval, HTTP calls, serialization and response streaming.
- Check for connection-pool waits, slow external APIs, retries, synchronous blocking and queue time before the database call.
- Compare server-side and client-side timings, including network and ingress, to locate hidden time.
- Fix the dominant span, then verify p95 under load and add stage-specific alerts.

PRACTICAL EXAMPLE

Example: trace_id -> auth 40 ms | pool wait 7 s | SQL 100 ms | external API 4.8 s.

INTERVIEW PERSPECTIVE

Distinguish DB execution time from connection acquisition and total request latency.

QUICK MEMORY TIP

12s TOTAL - 0.1s SQL = FIND 11.9s

AVOID

Assuming the database is responsible because the endpoint queries a database.

Q3. Your Python service occasionally runs out of memory while processing large PDFs. How would you investigate and fix it?

HOW I WOULD ANSWER

- Reproduce with representative large/scanned PDFs and inspect RSS, pod OOMKilled events and peak per-worker memory.
- Profile allocations with tracemalloc/memray; check page rendering, OCR images, copies, dataframes and retained references.
- Process pages incrementally, cap image resolution and worker concurrency, and release temporary buffers.
- Move heavy OCR/parsing to bounded background workers; set memory limits based on measured peaks and test worst-case documents.

PRACTICAL EXAMPLE

Example: kubectl describe pod <pod>; inspect OOMKilled, then profile peak memory per PDF page.

INTERVIEW PERSPECTIVE

Explain memory per concurrent job, not only memory per document; consider worker isolation.

QUICK MEMORY TIP

PROFILE > STREAM > LIMIT > VERIFY

AVOID

Reading all pages and rasterized images into memory at once.

Q4. Multiple users upload documents simultaneously and some processing jobs fail. How would you redesign the workflow?

HOW I WOULD ANSWER

- Accept uploads quickly into object storage; create a job record and return HTTP 202 with a job ID.
- Publish a durable queue message; background workers perform extraction, chunking and indexing with controlled concurrency.
- Make each stage idempotent; add retries with backoff, dead-letter queues, status tracking and poison-message handling.
- Monitor queue depth, oldest-message age, processing success and time-to-completion; autoscale workers separately.

PRACTICAL EXAMPLE

Example: FastAPI -> Azure Blob -> Service Bus -> worker -> PostgreSQL status.

INTERVIEW PERSPECTIVE

Discuss at-least-once delivery, duplicate processing, job recovery and user-visible progress.

QUICK MEMORY TIP

STORE > QUEUE > WORK > TRACK

AVOID

Running long OCR tasks inside the request-response path.

Q5. An API sometimes creates duplicate records when clients retry requests. How would you implement idempotency?

HOW I WOULD ANSWER

- Require an Idempotency-Key for mutation requests and scope it to the caller and operation.
- Atomically reserve the key and store request fingerprint, status and final response in durable storage.
- On retry: return the previous result; reject the same key with a different payload; guard races with unique constraints/transactions.
- For downstream side effects, use an outbox pattern or downstream idempotency keys; define retention and failure semantics.

PRACTICAL EXAMPLE

Example: UNIQUE(tenant_id, idempotency_key); POST /payments with repeat key returns same outcome.

INTERVIEW PERSPECTIVE

Explain concurrent duplicate requests and partial failure, not just client retries.

QUICK MEMORY TIP

KEY > ATOMIC SAVE > REPLAY

AVOID

Checking for duplicates in application memory only.

SCENARIO 06 / BACKEND DEVELOPMENT & APIs

Q6. Your backend has 20 tightly coupled modules and every release breaks another feature. How would you refactor it?

HOW I WOULD ANSWER

- Map dependencies and change hotspots; identify domain boundaries and risky shared state.
- Introduce clear API/service/repository boundaries with dependency injection; keep behavior stable with characterization tests.
- Refactor one vertical slice at a time behind interfaces/feature flags, then remove obsolete coupling.
- Use CI contract tests, code owners, architecture checks and deployment metrics to prevent regression.

PRACTICAL EXAMPLE

Example: route -> domain service -> repository; avoid direct cross-module DB access.

INTERVIEW PERSPECTIVE

Describe migration risk, team ownership, testing strategy and measurable improvement.

QUICK MEMORY TIP

MAP > TEST > SLICE > ENFORCE

AVOID

Rewriting all 20 modules in one release.

Q7. An asynchronous FastAPI endpoint still blocks other requests. What would you inspect?

HOW I WOULD ANSWER

- Inspect async endpoints for time.sleep, requests, synchronous SDK calls, CPU-heavy parsing and blocking ORM drivers.
- Measure event-loop lag and traces under concurrency; check threadpool saturation and sync dependency behavior.
- Use await with async-compatible I/O; run blocking I/O in a bounded threadpool and CPU-heavy work in processes/workers.
- Set timeouts, concurrency limits and load-test to confirm responsiveness under contention.

PRACTICAL EXAMPLE

Example: replace time.sleep(2) with await asyncio.sleep(2); use an async HTTP client.

INTERVIEW PERSPECTIVE

Explain the difference between I/O-bound and CPU-bound tasks and Python worker models.

QUICK MEMORY TIP

ASYNC I/O; OFFLOAD BLOCKING; ISOLATE CPU

AVOID

Assuming async def automatically makes every function non-blocking.

Q8. Your application needs to process a 1 GB document without exceeding pod memory limits. What architecture would you choose?

HOW I WOULD ANSWER

- Stream the 1 GB upload to Azure Blob or another object store; avoid buffering the entire body in the API pod.
- Use a durable job queue and worker that reads bounded chunks/pages; avoid full-document rasterization.
- Set per-job memory and time limits; split pages where possible and checkpoint progress for retries.
- Apply file-type validation, size limits, malware scanning where required, and monitor peak RSS and completion rate.

PRACTICAL EXAMPLE

Example: upload -> Blob -> Service Bus -> isolated PDF worker -> chunks -> vector index.

INTERVIEW PERSPECTIVE

Show capacity math: peak memory per worker × concurrent workers plus runtime overhead.

QUICK MEMORY TIP

STREAM > QUEUE > CHUNK > CHECKPOINT

AVOID

Increasing pod memory without bounding per-job allocations.

Q9. A critical dependency fails intermittently. How would you implement timeouts, retries, circuit breakers, and fallback behavior?

HOW I WOULD ANSWER

- Define connection and total timeouts for each dependency based on your request latency budget.
- Retry only transient and safe operations with exponential backoff, jitter and a capped retry budget.
- Use circuit breakers to stop cascading failures; apply bulkheads, concurrency limits and fallback/degraded responses.
- Observe timeout rate, retry amplification, circuit state and customer impact; test failure injection.

PRACTICAL EXAMPLE

Example: 300 ms connect timeout; capped retry on 503 for idempotent GET, not blind POST.

INTERVIEW PERSPECTIVE

Distinguish retryable from non-retryable errors; discuss idempotency and retry storms.

QUICK MEMORY TIP

TIMEOUT > SAFE RETRY > BREAKER > FALLBACK

AVOID

Retrying every error indefinitely, amplifying an outage.

SCENARIO 10 / BACKEND DEVELOPMENT & APIs

Q10. Two engineers disagree about microservices versus a modular monolith. How would you decide?

HOW I WOULD ANSWER

- Clarify independent scaling, deployment cadence, domain ownership, team size, data boundaries and availability needs.
- A modular monolith offers simpler transactions, debugging and operations when teams and boundaries are still evolving.
- Microservices help when domains require independent release/scaling and teams can own networking, observability and data consistency.
- Run a decision matrix or small spike; document ADRs and revisit when measurable triggers emerge.

PRACTICAL EXAMPLE

Example: keep one FastAPI service initially; extract ingestion workers when scaling and ownership diverge.

INTERVIEW PERSPECTIVE

Present trade-offs rather than treating microservices as inherently more advanced.

QUICK MEMORY TIP

TEAM > BOUNDARIES > SCALE > OPERATIONS

AVOID

Choosing microservices because the system uses AI or Kubernetes.

QUICK REVISION

The Lead AI Engineer answer framework

01 CLARIFY

Users affected, workload, p95 SLO, business impact.

02 MEASURE

Traces, latency stages, CPU/memory, queues, errors.

03 ISOLATE

Find the bottleneck before changing infrastructure.

04 DESIGN

Explain alternatives, trade-offs and failure modes.

05 VALIDATE

Load tests, regression checks, alerts and rollback.

06 LEAD

Ownership, incident communication, prevention.